

파머완 08 텍스트 분석

01 텍스트 분석 이해

텍스트 분석 ?

비정형 데이터인 텍스트 ⇒

1. 어떻게 **피처 형태**로 추출하는가?
2. 어떻게 추출된 피처에 **의미 있는 값**을 부여하는가?

피처 벡터화 or 피처 추출

1. 텍스트를 word 기반의 다수의 피처로 추출
2. 피처에 숫자 값 부여 (ex 단어 빈도수)
⇒ 단어의 조합인 벡터값으로 표현됨
- 방식: BOW (Bag of Words), Word2Vec

✅ 텍스트를 벡터값을 가지는 피처로 변환하는 것: ML 모델을 적용하기 전에 수행해야할 중요한 요소!

텍스트 분석 수행 프로세스

1. 텍스트 전처리
2. 피처 벡터화/추출: BOW - Count 기반, TF-IDF 기반 벡터화
3. ML 모델 수립 및 학습/예측/평가

파이썬 기반의 NLP, 텍스트 분석 패키지

: 텍스트 전처리, 피처 벡터화/추출, ML 모델을 지원하는 라이브러리가 많음

- NLTK

- Gensim
 - Spacy
-

02 텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

텍스트 정규화

: 텍스트를 ML, NLP에 입력 데이터로 사용하기 위해,
클렌징, 정제, 토큰화, 어근화 등의 텍스트 데이터의 사전 작업을 수행하는 것

클렌징

: 분석에 방해가 되는 불필요한 문자, 기호 등을 제거
ex) HTML, XML 태그 or 특정 기호

텍스트 토큰화

문장 토큰화

: 문장의 마지막을 뜻하는 기호에 따라 분리

ex) 문장의 마침표, 개행 문자

- 정규 표현식에 따른 토큰화도 가능
- 각 문장이 가진 시맨틱적 의미가 중요한 요소로 사용될 때 적용함

sent_tokenize() 토큰화 수행

```
from nltk import sent_tokenize
import nltk

text_sample='The Matrix is every where its all around us, here even in this roo
sentences=sent_tokenize(text=text_sample)
print(type(sentences),len(sentences))
print(sentences)
```

output)

```
<class 'list'> 3
```

```
['The Matrix is every where its all around us, here even in this room.', 'You can see it out your window or on your television.', 'You feel it when you go to work, or go to church or pay your taxes.']
```

: 문장으로 구성된 list 객체 반환

단어 토큰화

: 문장을 단어로 토큰화

ex) 공백, 콤마, 마침표, 개행문자 or 정규 표현식을 이용하여 다양한 유형으로 토큰화 수행

- 단어의 순서가 중요하지 않은 경우, 단어 토큰화만 사용

word_tokenize() 단어 토큰화 수행

```
from nltk import word_tokenize
```

```
sentence='The Matrix is everywhere its all around us, here even is this room.'
words=word_tokenize(sentence)
print(type(words),len(words))
print(words)
```

output)

```
<class 'list'> 15
```

```
['The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'is', 'this', 'room', '.']
```

sent_tokenize & word_tokenize 조합 토큰화 수행

```
from nltk import word_tokenize,sent_tokenize
```

```
# 여러 개의 문장으로 된 입력 데이터를 문장별로 단어 토큰화하게 만드는 함수 생성
def tokenize_text(text):
```

```
    # 문장별로 분리 토큰
```

```
    sentences=sent_tokenize(text)
```

```
    # 분리된 문장별 단어 토큰화
```

```
    word_tokens=[word_tokenize(sentence) for sentence in sentences]
```

```
    return word_tokens
```

```
# 여러 문장에 대해 문장별 단어 토큰화 수행
word_tokens=tokenize_text(text_sample)
print(type(word_tokens),len(word_tokens))
print(word_tokens)
```

output)

```
<class 'list'> 3
```

```
[['The', 'Matrix', 'is', 'every', 'where', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'in', 'this', 'room', '.'], ['You', 'can', 'see', 'it', 'out', 'your', 'window', 'or', 'on', 'your', 'television', '.'], ['You', 'feel', 'it', 'when', 'you', 'go', 'to', 'work', ',', 'or', 'go', 'to', 'church', 'or', 'pay', 'your', 'taxes', '.']]
```

word_tokens: 3개의 리스트 객체를 내포하는 리스트

- 개별 리스트 객체: 문장별로 토큰화된 단어를 요소로 가짐

n-gram

: 연속된 n개의 단어를 하나의 토큰화 단위로 분리해냄

- 문장을 단어별로 토큰화 한다면 → 문맥적 의미가 무시되므로 사용

스톱 워드 제거

스톱 워드: 분석에 큰 의미가 없는 단어

ex) is, the, a, will

- 언어별로 스톱 워드가 목록화 됨

스톱워드 필터링

```
import nltk

stopwords=nltk.corpus.stopwords.words('english')
all_tokens=[]
# 위 예제에서 3개의 문장별로 얻은 word_tokens list에 대해 스톱 워드를 제거하는 반복
for sentence in word_tokens:
    filtered_words=[]
    # 개별 문장별로 토큰화된 문장 list에 대해 스톱 워드를 제거하는 반복문
```

```

for word in sentence:
    # 소문자로 모두 변환
    word=word.lower()
    # 토큰화된 개별 단어가 스톱 워드의 단어에 포함되지 않으면 word_tokens에 추가
    if word not in stopwords:
        filtered_words.append(word)
    all_tokens.append(filtered_words)
print(all_tokens)

```

output)

```

[['matrix', 'every', 'around', 'us', ',', 'even', 'room', '.'], ['see', 'window', 'television', '.'], ['feel', 'go', 'work', ',', 'go',
'church', 'pay', 'taxes', '.']]

```

Stemming과 Lemmatization

: 문법적 / 의미적으로 변하는 단어의 원형을 찾음

Lemmatization

- 의미론적 기반으로 원형을 찾음
- 문법적 요소 + 더 의미적인 부분 ⇒ 정확한 철자로 된 어근 단어를 찾음
- 더 정교하고, 시간이 오래 걸림
-

Stemming

- 원형 단어로 변환 시 일반적인 방법 적용
- 더 단순화된 방법을 적용해 원래 단어에서 일부 철자가 훼손된 어근 단어를 추출하는 경향

LancasterStemmer() → Stemmer 수행

```

from nltk.stem import LancasterStemmer
stemmer=LancasterStemmer()

print(stemmer.stem('working'),stemmer.stem('works'),stemmer.stem('worked')

```

```
print(stemmer.stem('amusing'),stemmer.stem('amuses'),stemmer.stem('amus
print(stemmer.stem('happier'),stemmer.stem('happiest'))
print(stemmer.stem('fancier'),stemmer.stem('fanciest'))
```

output)

work work work

amus amus amus

happy happiest

fant fanciest

WordNetLemmatizer → Lemmatization 수행

- 단어의 품사를 입력해야함

```
from nltk.stem import WordNetLemmatizer
import nltk
```

```
lemma=WordNetLemmatizer()
print(lemma.lemmatize('amusing','v'),lemma.lemmatize('amuses','v'),lemma.le
print(lemma.lemmatize('happier','a'),lemma.lemmatize('happiest','a'))
print(lemma.lemmatize('fancier','a'),lemma.lemmatize('fanciest','a'))
```

output)

amuse amuse amuse

happy happy

fancy fancy

⇒ Stemmer 보다 정확한 추출

03 Bag of Words - BOW

BOW

: 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 단어에 대해 빈도 값을 부여해 피쳐 값을 추출하는 모델

피쳐 추출 과정

1. 문장 1,2의 모든 단어에서 중복을 제거하고 각 단어를 칼럼 형태로 나열 ⇒ 각 단어 고유의 인덱스 부여
2. 개별 문장에서 해당 단어가 나타나는 횟수를 각 단어에 기재함

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남

단점

- 문맥 의미 반영 부족: 단어의 순서 고려 X ⇒ 문맥적 의미 무시
- 희소 행렬 문제: 문서가 많을수록, 대부분의 데이터가 0값으로 채워짐

BOW 피쳐 벡터화

피쳐 벡터화

: 텍스트를 특정 의미를 가지는 숫자형 값인 벡터로 변환

카운트 기반의 벡터화

- 카운트 값이 높을수록 중요한 단어
- 문서의 특징보단, 언어 특성상 자주 사용되는 단어까지 높은 값 부여

TF-IDF 기반의 벡터화

: 개별 문서에서 자주 나타나는 단어에 높은 가중치

+ 전반적으로 자주 나타나는 단어에 페널티 부여



한 개의 문서(Document)



모든 문서들(Corpus)

Term Frequency

The	Matrix	is	nothing	but	an	advertising	gimmick
40	5	50	12	20	45	3	2

Document Frequency

The	Matrix	is	nothing	but	an	advertising	gimmick
2000	190	2300	500	1200	3000	52	12

$$TFIDF_i = TF_i * \log \frac{N}{DF_i}$$

- TF_i =개별 문서에서의 단어 i 빈도
- DF_i =단어 i 를 갖는 문서 수
- N =전체 문서 개수

사이킷런의 Count 및 TF-IDF 벡터화 구현

CountVectorizer

: 카운트 기반 벡터화 구현 클래스

- 피처 벡터화 + 텍스트 전처리 (소문자 일괄 변환, 토큰화, 스톱워드 필터링 등)
- `fit()`, `transform()` 을 통해 피처 벡터화된 객체 반환

입력 파라미터

<code>max_df</code>	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
<code>min_df</code>	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
<code>max_features</code>	추출하는 피처의 개수를 제한

stop_words	지정하는 언어의 스톱워드는 추출에서 제외
n_gram_range	n_gram 범위를 설정
analyzer	피처 추출을 수행한 단위를 지정
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용

step

1. 사전 데이터 가공: 모든 문자를 소문자로 변경하는 등의 전처리 작업
2. 단어 기준으로 n_gram_range를 반영해 각 단어 토큰화
3. 텍스트 정규화 수행
4. 피처 벡터화

BOW 벡터화를 위한 희소 행렬

사이킷런의 CountVectorizer/TfidfVectorizer

⇒ 텍스트를 피처 단위로 벡터화해 변환 / CSR 형태의 희소 행렬을 반환

희소 행렬 - COO 형식

: 0이 아닌 데이터만 별도의 배열에 저장

& 그 데이터가 가리키는 행과 열의 위치를 별도의 배열에 저장

넘파이 객체 생성

```
import numpy as np
dense=np.array([[3,0,1],[0,2,0]])
```

coo_matrix(): COO 형식의 희소 행렬 변환

```

from scipy import sparse

# 0이 아닌 데이터 추출
data=np.array([3,1,2])

# 행 위치와 열 위치를 각각 배열로 생성
row_pos=np.array([0,0,1])
col_pos=np.array([0,2,1])

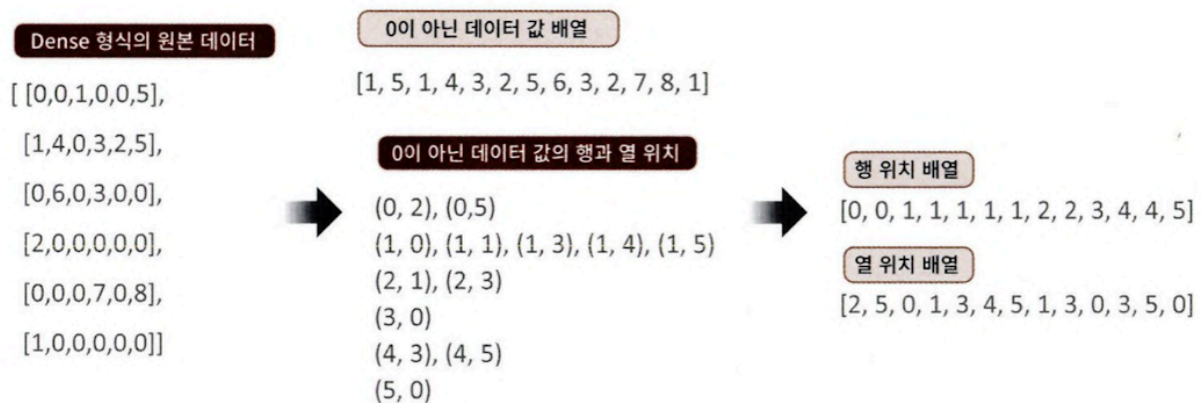
# sparse 패키지의 coo_matrix를 이용해 COO 형식으로 희소 행렬 생성
sparse_coo=sparse.coo_matrix((data,(row_pos,col_pos)))

```

희소 행렬 - CSR 형식

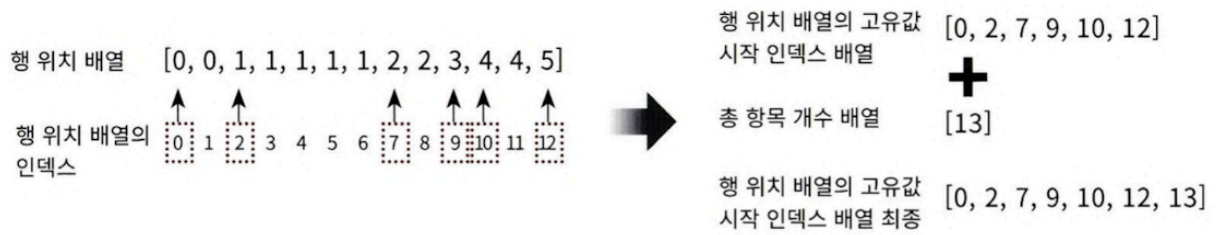
: COO 형식이 반복적인 위치 데이터를 사용하는 점을 해결한 방식

COO 형식 문제점



- 순차적인 같은 값이 반복적으로 나타남
 ⇒ 행 위치 배열이 0부터 순차적으로 증가하는 값으로 이뤄졌다는 특성을 고려하면, 행 위치 배열의 고유한 값의 시작 위치만 표기하는 방법으로 이러한 반복을 제거할 수 있음!

CSR 변환 방식



- 고유 값의 시작 위치만 알고 있으면, 행 위치 배열을 다시 만들 수 있음

csr_matrix()

```
from scipy import sparse

dense2=np.array([[0,0,1,0,0,5],[1,4,0,3,2,5],[0,6,0,3,0,0],[2,0,0,0,0,0],[0,0,0,7,0,0]])

# 0이 아닌 데이터 추출
data2=np.array([1,5,1,4,3,2,5,6,3,2,7,8,1])

# 행 위치와 열 위치를 각각 array로 생성
row_pos=np.array([0,0,1,1,1,1,2,2,3,4,4,5])
col_pos=np.array([2,5,0,1,3,4,5,1,3,0,3,5,0])

# COO 형식으로 변환
sparse_coo=sparse.coo_matrix((data2,(row_pos,col_pos)))

# 행 위치 배열의 고유한 값의 시작 위치 인덱스를 배열로 생성
row_pos_ind=np.array([0,2,7,9,10,12,13])

# CSR 형식으로 변환
sparse_csr=sparse.csr_matrix((data2,col_pos,row_pos_ind))
```

05 감성 분석

감성 분석 소개

: 문서의 주관적인 감성/의견/감정/기분 등을 파악하기 위한 방법

- 소셜 미디어, 여론조사, 온라인 리뷰, 피드백 등 다양한 분야에서 활용
- 문서 내 텍스트가 나타내는 여러 가지 주관적인 단어와 문맥을 기반으로 감성 수치를 계산
 - ⇒ 이러한 감성 지수는 긍정 감성 지수와 부정 감성 지수로 구성되며 이들 지수를 합산해 긍정 감성 또는 부정 감성을 결정

지도학습 / 비지도학습 방식

- 지도학습: 학습 데이터와 타깃 레이블 값을 기반으로 감성 분석 학습수행 → 이를 기반으로 다른 데이터의 감성 분석을 예측하는 방법
- 비지도학습: 'Lexicon'이라는 일종의 감성 어휘 사전 이용
 - Lexicon: 감성 분석을 위한 용어와 문맥에 대한 다양한 정보를 가지고 있으며 이를 이용해 문서의 긍정적, 부정적 감성 여부를 판단

지도학습 기반 감성 분석 실습 - IMDB 영화평

 문자열 삭제 → replace()

```
import re

# <br> html 태그는 replace 함수로 공백으로 변환
review_df['review']=review_df['review'].str.replace('<br/>',' ')

# 파이썬의 정규 표현식 모듈인 re를 이용해 영어 문자열이 아닌 문자는 모두 공백으로 변환
review_df['review']=review_df['review'].apply(lambda x:re.sub('[^a-zA-Z]',' ',x))
```

Count 벡터화 예측 성능

```
# 스톱 워드는 English, filtering, ngram은 (1,2)로 설정해 CountVectorization수행.
# LogisticRegression의 C는 10으로 설정.
pipeline = Pipeline([('cnt_vect', CountVectorizer(stop_words='english', ngram_

# Pipeline 객체를 이용하여 fit(), predict()로 학습/예측 수행. predict_proba()는 roc_
pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
```

```
pred_probs = pipeline.predict_proba(X_test['review'][:,1])
```

```
print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, p  
roc_auc_score(y_test, pred_probs)))
```

예측 정확도는 0.8868, ROC-AUC는 0.9505

TF-IDF 벡터화 예측 성능

```
# 스톱 워드는 english, filtering, ngram은 (1,2)로 설정해 TF-IDF 벡터화 수행.  
# LogisticRegression의 C는 10으로 설정.  
pipeline = Pipeline([('tfidf_vect', TfidfVectorizer(stop_words='english', ngram_  
  
pipeline.fit(X_train['review'], y_train)  
pred = pipeline.predict(X_test['review'])  
pred_probs = pipeline.predict_proba(X_test['review'][:,1])  
  
print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, p
```

예측 정확도는 0.8932, ROC-AUC는 0.9600

비지도학습 기반 감성 분석 소개

감성 지수

: 긍정 감성 또는 부정 감성의 정도를 의미하는 수치

- 단어의 위치나 주변 단어, 문맥, POS(Part of Speech) 등을 참고해 결정
- NLTK 패키지 - Lexicon 모듈 포함

WordNet

: 시맨틱 분석을 제공하는 어휘 사전

- 다양한 상황에서 같은 어휘라도 다르게 사용되는 어휘의 시맨틱 정보 제공

SentiWordNet을 이용한 감성 분석

synsets(): 지정된 단어의 모든 synset 객체 반환

```
from nltk.corpus import wordnet as wn

term = 'present'

# 'present'라는 단어로 wordnet의 synsets 생성.
synsets = wn.synsets(term)
print('synsets() 반환 type:', type(synsets))
print('synsets() 반환 값 갯수:', len(synsets))
print('synsets() 반환 값:', synsets)
```

path_similarity(): 단어 간 상호 유사도

```
# synset 객체를 단어별로 생성
tree = wn.synset('tree.n.01')
lion = wn.synset('lion.n.01')
tiger = wn.synset('tiger.n.02')
cat = wn.synset('cat.n.01')
dog = wn.synset('dog.n.01')

entities = [tree, lion, tiger, cat, dog]
similarities = []
entity_names = [entity.name().split('.')[0] for entity in entities]

# 단어별 synset 들을 iteration 하면서 다른 단어들의 synset과 유사도를 측정
for entity in entities:
    similarity = [round(entity.path_similarity(compared_entity), 2) for compared_entity in entities]
    similarities.append(similarity)

# 개별 단어별 synset과 다른 단어의 synset과의 유사도를 DataFrame형태로 저장
similarity_df = pd.DataFrame(similarities, columns=entity_names, index=entity_names)
similarity_df
```

	tree	lion	tiger	cat	dog
tree	1.00	0.07	0.07	0.08	0.12
lion	0.07	1.00	0.33	0.25	0.17
tiger	0.07	0.33	1.00	0.25	0.17
cat	0.08	0.25	0.25	1.00	0.20
dog	0.12	0.17	0.17	0.20	1.00

senti_synsets(): synsets()와 비슷하게 Senti_Synset 클래스를 리스트 형태로 반환

father, fabulous 감정 지수와 객관성 지수

```
import nltk
from nltk.corpus import sentiwordnet as swn

father = swn.senti_synset('father.n.01')
print('father 긍정감성 지수: ', father.pos_score())
print('father 부정감성 지수: ', father.neg_score())
print('father 객관성 지수: ', father.obj_score())
print('\n')
fabulous = swn.senti_synset('fabulous.a.01')
print('fabulous 긍정감성 지수: ', fabulous.pos_score())
print('fabulous 부정감성 지수: ', fabulous.neg_score())
```

```
father 긍정감성 지수: 0.0
father 부정감성 지수: 0.0
father 객관성 지수: 1.0
```

```
fabulous 긍정감성 지수: 0.875
fabulous 부정감성 지수: 0.125
```

SentiWordNet을 이용한 영화 감상평 감성 분석

step

1. 문서를 문장 단위로 분해

2. 다시 문장을 단어 단위로 토큰화하고 품사 태깅
3. 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
4. senti_synset 객체에서 긍정 감성/ 부정 감성 지수를 구하고 이를 모두 합산해 특정 임계치 값 이상일 때 긍정 감성으로, 그렇지 않을 때는 부정 감성으로 결정

WordNet: 단어로 토큰화 → 어근 추출 / 품사 태깅 추출

품사 태깅 함수 정의

```
from nltk.corpus import wordnet as wn

# 간단한 NLTK PennTreebank Tag를 기반으로 WordNet기반의 품사 Tag로 변환
def penn_to_wn(tag):
    if tag.startswith('J'):
        return wn.ADJ
    elif tag.startswith('N'):
        return wn.NOUN
    elif tag.startswith('R'):
        return wn.ADV
    elif tag.startswith('V'):
        return wn.VERB
    return
```

swn_polarity() 함수 생성

: 품사 태깅 후 SentiSynset 클래스 생성, polarity score 합산

```
def sw_n_polarity(text):
    # 감성 지수 초기화
    sentiment = 0.0
    tokens_count = 0

    lemmatizer = WordNetLemmatizer()
    raw_sentences = sent_tokenize(text)
    # 분해된 문장별로 단어 토큰 -> 품사 태깅 후에 SentiSynset 생성 -> 감성 지수 합산
```



```

for raw_sentence in raw_sentences:
    # NTLK 기반의 품사 태깅 문장 추출
    tagged_sentence = pos_tag(word_tokenize(raw_sentence))
    for word, tag in tagged_sentence:

        # WordNet 기반 품사 태깅과 어근 추출
        wn_tag = penn_to_wn(tag)
        if wn_tag not in (wn.NOUN, wn.ADJ, wn.ADV):
            continue
        lemma = lemmatizer.lemmatize(word, pos=wn_tag)
        if not lemma:
            continue
        # 어근을 추출한 단어와 WordNet 기반 품사 태깅을 입력해 Synset 객체를 생성
        synsets = wn.synsets(lemma, pos=wn_tag)
        if not synsets:
            continue
        # sentiwordnet의 감성 단어 분석으로 감성 synset 추출
        # 모든 단어에 대해 긍정 감성 지수는 +로 부정 감성 지수는 -로 합산해 감성 지수
        synset = synsets[0]
        swn_synset = swin.senti_synset(synset.name())
        sentiment += (swn_synset.pos_score() - swin_synset.neg_score())
        tokens_count += 1

    if not tokens_count:
        return 0

    # 총 score가 0 이상일 경우 긍정(Positive) 1, 그렇지 않을 경우 부정(Negative) 0 빈
    if sentiment >= 0:
        return 1

    return 0

```

- 각 단어의 긍정/부정 감성 지수 총합 > 0 : 긍정 감성 예측

VADER를 이용한 감성 분석

VADER

: 소셜 미디어의 감성 분석 용도로 만들어진, 룰 기반의 Lexicon

- SentimentIntensityAnalyzer 클래스 이용